Cyclic perturbations encode memories in trained neural networks without loss of performance

Max Kerr Winter[1,2] and Liesbeth M. C. Janssen[3]
Vrije Universiteit Amsterdam[1]
AMOLF[2]
Eindhoven University of Technology[3]
(Dated: October 26, 2026)

Memory formation under repeated driving is a defining collective behavior of many disordered materials. Whether the same mechanism can operate in artificial neural networks, whose parameters also evolve on a high-dimensional landscape, is unknown. Here we show that cyclically perturbing the weights of a trained, overparameterized deep neural network encodes the perturbation amplitude in its parameters. During each cycle, all weights are displaced and the network is allowed to relax by gradient descent. Repetition organizes the network into a state that is reversible for cycles no larger than the training amplitude. The encoded value can therefore be read from the onset of irreversible weight motion as the probe amplitude is increased. Multiple amplitudes can also be encoded, although only the largest memory persists at long times under deterministic driving. Guided by an analogy with cyclically sheared suspensions, we introduce a noisy loss function that preserves all memories at long times. Strikingly, neither deterministic nor noisy memory writing measurably changes performance on unseen classification data. These results identify cyclic memory as a behavior shared by particulate matter and neural networks, and show that information can be stored in the geometry of a network's low-loss regions without compromising its original function.

I. SIGNIFICANCE STATEMENT

Materials can remember how strongly they have been driven: repeated deformation organizes their microscopic components so that a previous deformation amplitude can later be recovered. We show that the same form of memory emerges in trained deep neural networks. Repeatedly perturbing and relaxing a network's weights encodes one or more amplitudes, which can be read from the network's response to subsequent perturbations. A noise-inspired protocol makes multiple encoded values persist, yet the network remains just as effective on its original classification task. These findings draw a surprising link between two disparate fields: memory formation in physical materials, and deep learning. This work uncovers a novel capability of deep neural networks and demonstrates that a trained network can acquire additional information without sacrificing its existing function.

II. INTRODUCTION

Memory effects - where the state or dynamics of a system depends on its history - play a significant role in a vast number of systems across the natural sciences and engineering. In recent years, an intriguing set of phenomena have emerged whereby memories (i.e. sequences of values) can be written and read from systems of disordered particles, without imposing any obvious order on them [1–4]. This striking result was highly novel given that established methods for recording information (written text, bits encoded on a hard drive) are very deliberately not disordered.

How does reading and writing memories in disordered systems work in practice? Taking the model of Keim and Nagel [1] as a canonical example, a random suspension of particles is subjected to repeated cyclic shear with amplitude A (i.e. the write amplitude, or memory). Initially, a shear cycle will result in many particle collisions. With each collision the particles rearrange, and hence fail to return to their original positions once the shear cycle is complete. However, after many shear cycles, the particles will be organised into positions where they no longer collide. Once all particles avoid collisions, the system exists in a steady state when viewed stroboscopically (i.e. once per cycle). This state encodes the write amplitude A. The read protocol consists of single shear cycles at varying amplitudes $\tilde{A}$, applied to copies of the system on which the memory has been written. When $\tilde{A} \leq A$, all particles return to their original positions at the end of the shear cycle. When $\tilde{A} > A$, collisions will start to occur hence some particles will be displaced. Consequently, A can be deduced from the inflection point on a plot of the number of displaced particles against $\tilde{A}$. Many variations on this basic set up have been studied, including reading/writing multiple transient memories [1, 2], memory formation in dense suspensions [5, 6], memory formation in adaptive networks [7], and the effect of random driving [8].

Deep neural networks (DNNs) appear far removed from particulate materials, yet the two systems share a many common aspects in their mathematical structure. Network weights are high-dimensional degrees of freedom, and the loss function defines a landscape on which training dynamics evolve. In this analogy, weights play the role of particle coordinates and the loss function plays the role of the energy [9–14]. The ratio P/N of training examples to network parameters has a role

resembling particle density. At low P/N, overparameterized networks can fit every training example and access extended regions of the loss landscape with training loss $\mathcal{L} = 0$. Increasing P/N produces an underparameterized regime with nonzero loss, analogous to the onset of interactions in a denser particle system [10].

However, the analogy between DNNs and disordered materials has important limits. Overparameterized networks have no established counterpart of particle caging, their zero-loss regions can extend over long distances, and stochastic gradient descent and its related training algorithms produce nonthermal noise that is anisotropic, has a non-flat power spectrum, and varies in intensity over time [14–23]. Furthermore, trained networks can generalize to unseen data - a phenomenon with no obvious physical counterpart. These differences make it unclear whether memory formation in disordered materials can be translated to a neural network, or whether the motion within a low-loss region required by disordered memory read/write protocols would preserve the computation learned by the network.

Here we demonstrate that trained DNNs can acquire readable memories under cyclic weight perturbations. A deterministic protocol stores one long-lived amplitude and a second transient amplitude. A modified noisy protocol stabilizes both amplitudes over the observation time. Throughout, the networks retain their performance on unseen data. The memories therefore occupy degrees of freedom that are available within a low-loss region while leaving the network's original predictive function intact. As such, this work constitutes an unforeseen generalisation of the disordered memory concept to a new class of systems, as well as identifying a novel and as yet unexploited capability of deep neural networks.

III. RESULTS

A. Cyclic driving produces a readable memory

As a proof of principle, we used the benchmark test of digit recognition from the standard MNIST dataset [24], and asked whether a network trained on this task could exhibit memory effects analogous to those seen in particulate systems. For simplicity, we restricted ourselves to a binary supervised classification task where fully connected networks were trained to distinguish the MNIST digits 2 and 6. The images were reduced to 10 principal components, following the approach of [10] and [14], and networks with three hidden layers were trained using the quadratic hinge loss. Figure 1B shows that as hidden-layer width increased, the final training loss dropped by several orders of magnitude, marking entry into the overparameterized regime. All subsequent experiments used width 10 networks (410 parameters) that achieved a final training loss below 10^{-8}.

Our goal was to imprint a memory value, A, onto neural networks that had already been trained on the above digit recognition task, in such a way that we could recover the memory later. To do so, we displaced every network weight in equal increments along a fixed direction until the cumulative displacement reached A. Gradient descent relaxed the network after every increment. This is the upward sweep depicted in Fig. 1C. We then reversed the displacement and incremented the weights downwards, again interspersing the increments with periods of gradient descent, as depicted in the downward sweep of the same figure. The process of writing a memory consisted of repeated applications of an upward sweep followed by a downward sweep. If the trajectory in weight space of an upward and downward sweep remains entirely on a flat $\mathcal{L} = 0$ region of the loss landscape, the network returns to its starting point. However, if the trajectory encounters a nonzero-loss region, gradient descent displaces it, and so the completed cycle no longer returns every weight to its initial value. Such an event is depicted schematically in Fig. 1D, and is analogous to a collision between particles during a cycle of shear in the case of memory writing in physical systems. Repeated write cycles thus move the network around in the loss landscape until a state is reached that is fully reversible up to the memory value, A.

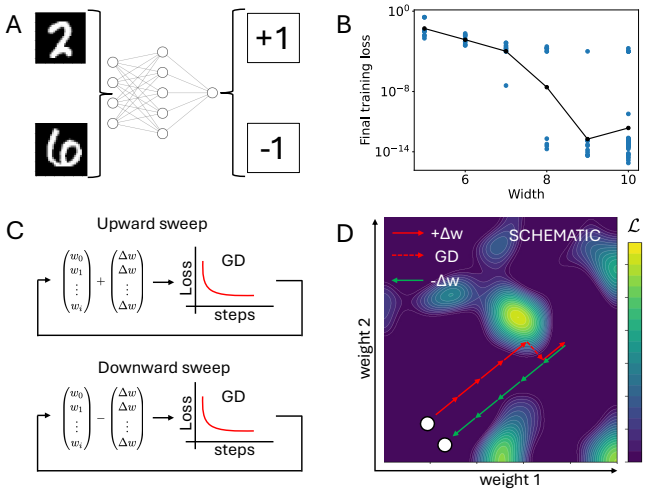


FIG. 1. (A) Networks classify MNIST digits 2 and 6, assigned labels +1 and −1, respectively. (B) Final training loss as a function of hidden-layer width. Networks begin to reach $\mathcal{L} \approx 0$ near width 8, identifying the underparameterized-to-overparameterized transition. Some wider networks remain at high loss because of their initial conditions and stochastic training trajectories. (C) The write protocol consists of upward and downward weight sweeps, with gradient-descent relaxation after each step. (D) Schematic trajectory through the loss landscape. The initial and final states are open circles; upward and downward sweeps are red and green, respectively. Relaxation on a non-flat region (dashed arrow) prevents return to the initial state.

We read the memory by applying one cycle of an upward and downward sweep to a copy of the written network at a probe amplitude $\tilde{A}$, and then probing across

many values of $\tilde{A}$. The readout is the number of weights displaced by more than a sensitivity ϵ after the read cycle. What does this look like in practice? Fig. 2A shows the number of displaced weights as a function of $\tilde{A}$, averaged over 41 independently trained networks, each of which has been subjected to the write protocol with memory value $A = 0.001$. The individual curves correspond to applying the read protocol after increasing numbers of write cycles. As the number of write cycles increases, a sharp inflection point emerges at $\tilde{A} = A$, demonstrating that the memory value has indeed been stored in the structure of the DNN, and can be successfully read by our protocol. The inflection point demarcates the boundary between reversible and irreversible weight trajectories under the read protocol.

The same read/write protocols can encode more than one value - a crucial step towards using cyclic memory in DNNs for any practical purposes. We alternated a cycle at $A_0 = 0.001$ with six cycles at $A_1 = 0.0005$. Once again applying the read protocol and plotting the result in Fig. 2B, both memory values appeared as changes in the slope of the read curves. As expected, the smaller amplitude was transient, and disappeared after approximately 140 write cycles. This hierarchy mirrors the behaviour of multiple transient memories in cyclically driven suspensions: repeated driving ultimately organizes the system around the largest amplitude, erasing signatures of smaller amplitudes [1, 2]. As such, the results in Fig. 2 exactly reproduce the phenomenology of particulate systems, and constitute a clear demonstration of the existence of cyclic memory in DNNs.

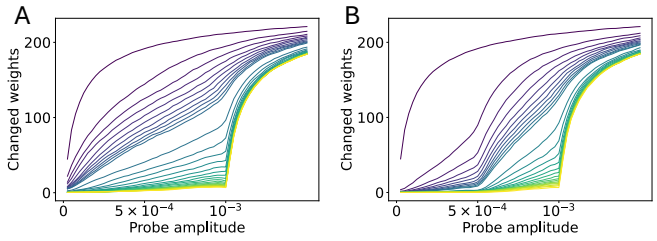


FIG. 2. Memories written by deterministic cyclic perturbations. (A) Number of weights displaced by more than $\epsilon = 5 \times 10^{-6}$ after a read cycle of amplitude $\tilde{A}$. A memory written at $A = 0.001$ appears at the onset of weight motion. (B) Memories of $A_0 = 0.001$ and $A_1 = 0.0005$ appear as changes in slope. The smaller memory is erased after approximately 140 write cycles. In both panels, color progresses from 0 write cycles (purple) to 200 cycles (yellow).

B. A novel noisy write protocol stabilizes multiple memories

A striking aspect of cyclic memory in particulate systems is that noise can, counterintuitively, stabilise multiple memories, preventing smaller memories from being erased by the largest one. The reason for this is that the noise continually returns particles to regions of space from which they were previously swept out [1]. Collisions correspond to the system moving out of a zero energy state, hence in the energy landscape picture, noise repeatedly drives the system onto the hills surrounding zero energy basins. It is not immediately obvious if noise will also preserve multiple memories in DNNs. In fact, we found that the most literal translation of this mechanism to DNNs, i.e. directly adding Gaussian noise to the network weights, did not reproduce the expected behavior. In fact, it produced the more naively intuitive result that memories were simply erased by the noise (Supporting Information). Instead, we modified the loss during the write cycles so that previously flat regions were assigned a weak, randomly varying gradient pushing the network toward the finite-loss boundaries (see Eq. 5 for the full definition). We call this modification "spring noise", due to its qualitative similarity to placing springs between randomly selected pairs of particles. The read protocol remained unchanged and deterministic.

The effect of introducing spring noise during the write cycles of a memory written at $A = 0.001$ is shown in Fig. 3A. As with the deterministic case in Fig. 2A, there is a large inflection at $\tilde{A} = A$, demonstrating that the memory has been successfully written and read. In contrast to Fig. 2A, some weights continued to move for $\tilde{A} < A$ after many write cycles, long past the point where the deterministic write protocol reaches a reversible state. This persistent subthreshold response is also seen in particles subjected to thermal noise, and is the network analog of continued particle collisions in previously swept out regions of space.

We next applied the write protocol with spring noise to the case of multiple memories, with values $A_0 = 0.001$ and $A_1 = 0.0005$, again applied in the same sequence as in the deterministic experiment of Fig. 2B. The spring noise result is shown in Fig. 3B, where both memories remained visible as abrupt gradient changes in the readout curves after 360 write cycles. The smaller memory therefore persisted for substantially longer than its deterministic counterpart, which was erased after approximately 140 cycles. Consequently, Fig. 3B demonstrates the effectiveness of spring noise at recovering the phenomenology of noisy particle suspensions. The existence of a write/read protocol capable of storing multiple persistent memories in DNNs is crucial for any practical application of this phenomenon.

C. Memory writing preserves predictive performance on unseen data

A major distinction between a neural network and a disordered material is that the network can perform a learned computation. This raises the important question of what effect, if any, writing and reading memories by cyclic perturbation has on the DNNs performance on the task it was originally trained to perform. We therefore

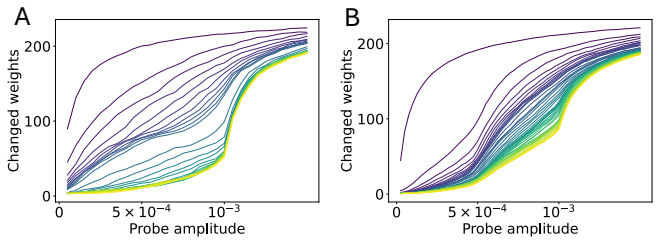


FIG. 3. Memories written with spring noise. (A) A memory at $A = 0.001$ remains visible, while weights continue to move for $\tilde{A} < A$. Color progresses from 0 write cycles (purple) to 200 cycles (yellow). (B) Memories at $A_0 = 0.001$ and $A_1 = 0.0005$ both remain visible after 360 cycles. Color progresses from 0 cycles (purple) to 360 cycles (yellow). Both panels use $\sigma = 10^{-7}$ and $\epsilon = 5 \times 10^{-6}$. See Eq. 5 for the definition of σ.

compared the the networks' loss values on unseen test data before and after writing a memory. As shown in Fig. 4, neither the deterministic protocol nor the spring noise protocol produced a discernible change in the distribution of test losses. Thus, within the amplitudes and times studied, cyclic driving reorganized the network weights enough to encode additional information without measurably degrading generalization on the original classification task. In other words, the predictive performance on unseen data is not compromised at all, and yet the DNNs have been enriched by additional memory capabilities.

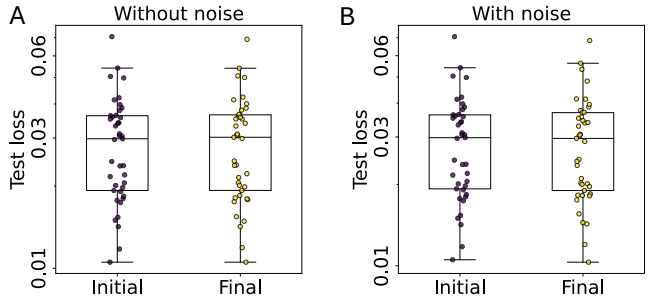


FIG. 4. Effect of memory writing on predictive performance. Test-loss distributions are shown before and after 200 cycles writing a memory at $A = 0.001$. (A) Deterministic write protocol. (B) Spring noise protocol with $\sigma = 10^{-7}$. Neither protocol produces a discernible change in test loss.

IV. DISCUSSION

Our results establish a form of memory in trained DNNs that is distinct from the associative memories of Hopfield networks and the persistent dynamical states of recurrent networks. A Hopfield memory corresponds to an attractor, so storage capacity is related to the number of stable minima [25, 26]. Here an amplitude is represented instead by the distance from the network's state to a finite-loss boundary along a prescribed perturbation path. Capacity is therefore controlled by the geometry of a low-loss region rather than by the number of minima.

This geometric distinction may help expose where the analogy between neural networks and disordered materials succeeds and where it fails. In particulate systems, zero-energy regions are constrained by nearby collisions and by boundary conditions. Such systems are typically isotropic, hence collisions occur when any particle moves in any direction by roughly one interparticle distance. By contrast, weights in an overparameterized network can diffuse for long times without encountering a barrier [14, 15]. Furthermore, the weights of our DNNs have free boundary conditions, hence Gaussian noise is likely to result in an effect similar to a gas expanding, with fewer and fewer interactions as the particles move apart.

These differences between DNNs and particles may explain why Gaussian weight noise did not stabilize multiple memories: unconstrained diffusion can carry the network away from finite-loss boundaries rather than repeatedly returning it to them. Spring noise was designed to restore that return and successfully recovered the phenomenology of thermally driven particle systems, despite being dissimilar to thermal fluctuations. The importance of basin shape does, however, suggest a potentially important difference between memory formation by cyclic perturbations in DNNs compared with physical systems. If DNNs do indeed have much larger basins, as suggested above, then it could be that they will have a much higher memory capacity than a comparable set of particles.

The preservation of test performance shows that movement within a low-loss region can alter a network's response to parameter perturbations without changing its measured predictive function. This is an important condition for establishing cyclic memory as a useful addition to established DNN functionality. Furthermore, it moves DNNs a small step closer to the biological scenario of information storage and processing occurring in the same structures, as opposed to the standard computer architecture separating the two.

Determining how many independent amplitudes or perturbation directions can be stored, how capacity scales with network size, and whether analogous memory phenomena exist in the underparameterized regime are immediate open questions. More broadly, the results suggest that memory formation by cyclic perturbation could be a quite general phenomenon in disordered systems, not just restricted to particles, or the charge-density waves which the particle work was inspired by [27, 28]. Perhaps the analogy could be extended even further afield, to disordered ecosystems [29] and beyond.

V. MATERIALS AND METHODS

A. Data and network preparation

We selected images of digits 2 and 6 from MNIST and assigned labels $y = +1$ and $y = -1$, respectively [24]. Principal-component analysis reduced each 28×28 image to 10 components. The training set contained 5,000 images and the test set contained 1,000 images, with equal numbers of each digit in both sets.

The models were fully connected, feed-forward networks without bias parameters. Each network had 10 input units, three hidden layers of equal width with ReLU activations, and one output unit. We varied hidden-layer width from 5 to 10 to locate the overparameterization transition. Networks were trained for 10,000 epochs by stochastic gradient descent with learning rate 0.01 and batch size 32. Memory experiments used all width-10 networks whose final training loss was below 10^{-8}.

For P training examples, the quadratic hinge loss was

$$\mathcal{L} = \frac{1}{P} \sum_{i=1}^{P} \ell(f_i, y_i), \tag{1}$$

$$\ell(f_i, y_i) = \frac{1}{2} \max\left(0, \Delta(y_i, f_i)\right)^2, \tag{2}$$

$$\Delta(y_i, f_i) = 1 - y_i f_i, \tag{3}$$

where f_i is the network output for example i. The flat zero of this loss provides extended regions in which perturbations can be reversible.

B. Deterministic write protocol

Starting from trained parameters θ, we added a scalar increment Δw to every weight, then performed $N_{\mathrm{GD}} = 2,000$ full-batch gradient-descent steps at learning rate $\eta = 0.01$. We repeated the displacement and relaxation $A/\Delta w$ times, then reversed the process by subtracting Δw before each relaxation. The upward and downward sweeps formed one write cycle. Unless stated otherwise, $\Delta w = 2.5 \times 10^{-5}$. Manual checks on several networks confirmed convergence within 2,000 relaxation steps.

Single-memory experiments repeated cycles with $A = 0.001$. For two-memory experiments, cycles were applied in the sequence

$$(A_0, A_1, A_1, A_1, A_1, A_1, A_1), \tag{4}$$

with $A_0 = 0.001$ and $A_1 = 0.0005$. Repeating the smaller amplitude was not required to create the memory but accelerated the evolution of the low-amplitude part of the read curve.

C. Read protocol

Each read amplitude $\tilde{A}$ was tested on a separate copy of the written network. We applied one upward and downward cycle using the same Δw, relaxation time, and learning rate as in the write protocol. The memory readout was the number of weights whose final values differed from their values before the read cycle by more than $\epsilon = 5 \times 10^{-6}$. Changes in the slope of this count as a function of $\tilde{A}$ identified written amplitudes.

D. Spring-noise protocol

During noisy writing, we replaced the per-example loss by

$$\ell(f_i, y_i) = \frac{1}{2} \max\left(\sqrt{m(f_i - y_i)}, \Delta(y_i, f_i)\right)^2, \tag{5}$$

where $m \sim \mathcal{N}(0, \sigma)$ and the remaining definition of $\mathcal{L}$ was unchanged. A new m was sampled after each increment or decrement of Δw. This replacement gives the nominally flat part of the loss a random gradient directed toward the quadratic region. We used $\sigma = 10^{-7}$ for the results in Fig. 3. All reads were performed without noise.

E. Gaussian weight-noise control

As a control, independent Gaussian random values were added directly to all weights after each increment or decrement during writing. At standard deviation 10^{-6}, a memory written at $A = 0.001$ retained a flat response below A (Fig. 5). Smaller noise had no visible effect, whereas larger noise removed the sharp response at A.

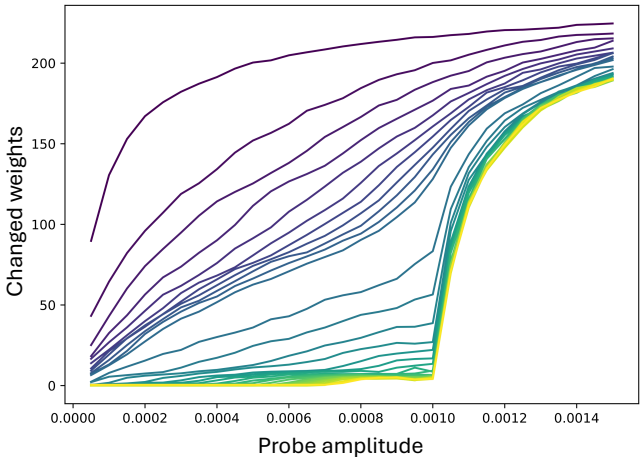


FIG. 5. Gaussian weight-noise control. The number of changed weights remains zero below the writing amplitude $A = 0.001$. Parameters are $\epsilon = 5 \times 10^{-6}$ and noise standard deviation 10^{-6}.

VI. ACKNOWLEDGMENTS

M.K.W. gratefully acknowledges funding from Stichting Ammodo. This work used the Dutch national e-infrastructure with the support of the SURF Cooperative using grant no. EINF-19321.

VII. AUTHOR CONTRIBUTIONS

M.K.W. and L.M.C.J. designed the research; M.K.W. performed the research and analyzed the data; M.K.W. and L.M.C.J. wrote the paper.

VIII. COMPETING INTEREST STATEMENT

The authors have no competing interests.

IX. DATA, MATERIALS, AND SOFTWARE AVAILABILITY

To be completed with repository and accession information before submission.

[1] N. C. Keim and S. R. Nagel, Physical Review Letters **107**, 1 (2011), arXiv:1101.2931.

[2] N. C. Keim, J. D. Paulsen, and S. R. Nagel, Physical Review E - Statistical, Nonlinear, and Soft Matter Physics **88**, 1 (2013), arXiv:1307.1184.

[3] N. C. Keim, J. D. Paulsen, Z. Zeravcic, S. Sastry, and S. R. Nagle, Reviews of Modern Physics **91**, 35002 (2019).

[4] J. D. Paulsen and N. C. Keim, Annual Review of Condensed Matter Physics **16**, 61 (2025), arXiv:2405.08158.

[5] D. Fiocco, G. Foffi, and S. Sastry, Physical Review Letters **112**, 1 (2014).

[6] D. Fiocco, G. Foffi, and S. Sastry, Journal of Physics Condensed Matter **27** (2015), 10.1088/0953-8984/27/19/194130.

[7] K. Bhattacharyya, D. Zwicker, and K. Alim, Physical Review Letters **129**, 28101 (2022).

[8] M. Mungan, D. Kumar, S. Patinet, and D. Vandembroucq, Physical Review Letters **134**, 178203 (2025).

[9] S. Franz and G. Parisi, Journal of Physics A: Mathematical and Theoretical **49**, 145001 (2016), arXiv:1501.03397.

[10] M. Geiger, S. Spigler, S. D'Ascoli, L. Sagun, M. Baity-Jesi, G. Biroli, and M. Wyart, Physical Review E **100**, 012115 (2019), arXiv:1809.09349.

[11] M. Baity-Jesi, L. Sagun, M. Geiger, S. Spigler, G. Ben Arous, C. Cammarota, Y. LeCun, M. Wyart, and G. Biroli, Journal of Statistical Mechanics: Theory and Experiment **2019**, 124013 (2019), arXiv:1803.06969.

[12] Y. Bahri, J. Kadmon, J. Pennington, S. S. Schoenholz, J. Sohl-Dickstein, and S. Ganguli, Annual Review of Condensed Matter Physics **11**, 501 (2020).

[13] D. Ghio, Y. Dandi, F. Krzakala, and L. Zdeborová, Proceedings of the National Academy of Sciences of the United States of America **121**, 1 (2024), arXiv:2308.14085.

[14] M. K. Winter and L. M. Janssen, Physical Review Research **7**, 23010 (2025), arXiv:2405.13098.

[15] B. L. Annesi, C. Lauditi, C. Lucibello, E. M. Malatesta, G. Perugini, F. Pittorino, and L. Saglietti, Physical Review Letters **131**, 227301 (2023), arXiv:2305.10623.

[16] N. Yang, C. Tang, and Y. Tu, Physical Review Letters **130**, 237101 (2023), arXiv:2206.01246.

[17] M. Kühn and B. Rosenow, arXiv , 20 (2023), arXiv:2306.05300.

[18] S. L. Smith, E. Elsen, and S. De, Proceedings of the 37th International Conference on Machine Learning (2020), arXiv:2006.15081.

[19] U. Şimşekli, L. Sagun, and M. Gurbuzbalaban, 36th International Conference on Machine Learning, **97**, 5827 (2019), arXiv:1901.06053.

[20] D. Soudry and Y. Carmon, arXiv (2016), arXiv:1605.08361.

[21] N. S. Keskar, J. Nocedal, P. T. P. Tang, D. Mudigere, and M. Smelyanskiy, 5th International Conference on Learning Representations, ICLR 2017 - Conference Track Proceedings , 1 (2017), arXiv:1609.04836.

[22] L. Wu, M. Wang, and W. J. Su, Advances in Neural Information Processing Systems **35** (2022), arXiv:2207.02628.

[23] B. Wu, T. Duan, and J. He, Computational Economics **51**, 743 (2018).

[24] "MNIST dataset, http://yann.lecun.com/exdb/mnist/,".

[25] J. J. Hopfield, Proceedings of the National Academy of Sciences of the United States of America **79**, 2554 (1982).

[26] D. J. Amit, H. Gutfreund, and H. Sompolinsky, Physical Review A **32**, 1007 (1985).

[27] S. N. Coppersmith, T. C. Jones, L. P. Kadanoff, A. Levine, J. P. McCarten, S. R. Nagel, S. C. Venkataramani, and X. Wu, Physical Review Letters **78**, 3983 (1997).

[28] M. L. Povinelli, S. N. Coppersmith, L. P. Kadanoff, S. R. Nagel, and S. C. Venkataramani, Physical Review E - Statistical Physics, Plasmas, Fluids, and Related Interdisciplinary Topics **59**, 4970 (1999).

[29] A. Altieri, F. Roy, C. Cammarota, and G. Biroli, Physical Review Letters **126**, 258301 (2021), arXiv:2009.10565.